

openvaf-r / OSDI Nuances and Gotchas

Non-obvious behaviors at the Verilog-A / OSDI to ngspice-netlist boundary

Ngspice + OpenVAF Enhancements project

July 2026

Contents

openvaf-r / OSDI nuances and gotchas	2
1. Unconnected trailing terminals are <i>optional</i> , not an error	3
2. Vector/bus ports flatten to <i>positional</i> terminals	4
3. Array/bus node names contain literal brackets	5
4. There is no "current between two nodes"	6
5. Two composition traps	7
Scope	8

openvaf-r / OSDI nuances and gotchas

A running list of non-obvious behaviors at the boundary between a Verilog-A model (compiled by **openvaf-r** to a **.osdi** library) and the ngspice netlist. None of these are bugs — they are consequences of how Verilog-A / OSDI semantics map onto a SPICE netlist — but each can surprise you, so they are collected here.

Companion reading: the [built-in natures and disciplines](#) reference and the [compiler internals](#) guide.

1. Unconnected trailing terminals are *optional*, not an error

A Verilog-A model port can be left unconnected on the instance line; the trailing terminal then becomes an internal node and the model sees it as open. This is the standard `$port_connected` mechanism (also how Spectre behaves), used e.g. for an optional self-heating dt port:

```
if ($port_connected(dt)) /* thermal node wired */ ;
else                     /* run isothermal */ ;
```

In ngspice's OSDI layer (`osdi/osdisetup.c`), each unconnected terminal is marked with a `-1` sentinel; `connected_terminals` is the index of the first `-1`, and everything from there on is allocated as an internal node and reported as *not connected* to `setup_instance` (so `$port_connected` returns false):

```
uint32_t connected_terminals = descr->num_terminals;
for (uint32_t i = 0; i < descr->num_terminals; i++)
    if (terminals[i] == -1) { connected_terminals = i; break; }
```

The consequence — an asymmetric, silent lower bound. For a device with N terminals:

nodes on the instance line	result
exactly N	runs normally
fewer than N (trailing)	runs silently — the missing trailing terminals become internal nodes; <code>\$port_connected</code> is false for them. <i>No warning</i> .
more than N	hard error: too many nodes connected to instance

So a genuine netlist typo that drops a trailing node looks identical to a deliberately-open optional port — it is accepted without a diagnostic. Classic SPICE primitives (R, C, a 4-terminal MOSFET) reject a short node list; OSDI trades that check for the optional-port feature. There is no per-port “optional” flag in Verilog-A (any port may be left open, that is what `$port_connected` is for), so a simulator cannot tell an intended open port from an accidental one. Mind your trailing terminal counts — the too-many case is caught for you, the too-few case is not.

2. Vector/bus ports flatten to *positional* terminals

A Verilog-A vector (bus) port ([Enhancement-3 / E-89](#)) such as output `out[0:3]`; compiles to *four separate positional terminals* in the .osdi device. The netlist connects them by position, one node per bit:

```
module tapbuf(in, out); output out[0:3]; ... // 5 terminals: in, out[0..3]
```

```
N1 in o0 o1 o2 o3 tmod ; in -> in, o0..o3 -> out[0..3]
```

There is no bus object in the netlist — only a flat, ordered node list. The array/bus node-range syntax ([Enhancement-221](#)) `a[0:3]` is a *pre-parse textual unroll* (`inp_expand_buses` in `frontend/inpcom.c`): it rewrites the single token `a[0:3]` into the four tokens `a[0]` `a[1]` `a[2]` `a[3]` wherever it appears, with no binding to any particular port. So it composes with a bus port naturally —

```
N1 in a[0:3] tmod ; -> N1 in a[0] a[1] a[2] a[3] tmod
```

— but the range carries no semantics: `N1 a[0:3] in tmod` is equally accepted and unrolls positionally to a *different* (here electrically singular) connection. The range is a typing shorthand; getting the positions right is on you (see also nuance 1 for the terminal-count behaviour). Confirmed example: with `tapbuf` above and $V_{in} = 2$ V , $v(a[k]) = 0.25 \cdot (k+1) \cdot 2 = 0.5 \cdot (k+1) \rightarrow 0.5, 1.0, 1.5, 2.0$.

3. Array/bus node names contain literal brackets

Because the range unroll produces node *names* with literal brackets, the node is called `a[0]` — the `[` and `]` are part of the name. In ngspice's interactive expression parser (`print` / `plot` / `let`), `[...]` is the vector-index operator, so `a[0]` naturally parses as “*element 0 of a vector a*”. [Enhancement-224](#) adds a literal-node fallback so `print a[0]` and `print v(a[0])` resolve the *node* when no vector `a` exists; ordinary vector indexing (`realvec[3]`) is unaffected. Good to know:

- `vec_get` / `vec_fromplot` (`frontend/vectors.c`) look names up literally (`findvec`) — they do *not* parse brackets; the index parsing is purely in the expression grammar.
- `vec_fromplot` also strips the `v(node)` / `i(node)` wrappers: `v(x)` → node `x`, and `i(x)` → `x#branch`.

4. There is no "current between two nodes"

ngspice has no `I(node1, node2)` branch-current syntax (that is Verilog-A / Spectre form). Branch current is read as a device parameter `@device[i]` or, for a voltage source, `i(Vsrc)`. For a two-terminal element written as a bus, e.g. a resistor `R1 a[0:1]` ($\rightarrow$ `R1 a[0] a[1]`), its current is `@r1[i]`, not `I(a[0], a[1])`. `i()` in an expression takes a *source name*, not a node pair.

5. Two composition traps

Both of these are invisible to single-feature testing, which is why they survived several fuzzing rounds.

A **case** inside a **do-while** used to crash the compiler (E-363). The MIR for that shape folds to a block whose terminator jumps to *itself*, and the CFG simplifier merged such a block into itself — retargeting its predecessors to itself (a no-op) and then deleting it, leaving terminators naming a block no longer in the layout. `if/else`, `while`, `for`, `repeat` and a nested `do-while` in the same position were always fine.

Array parameters and instantiation (E-363). A module has three array collections — buses, array variables and array *parameters* — and elaboration renamed only the first two per instance. So a module with an array parameter could not be instantiated twice, and two different modules sharing an array-parameter name collided too.

Still open: a provably non-terminating analog loop fails to compile. After the fix above its MIR is well formed, but everything after the loop is unreachable, so codegen reads values no reachable block defines. There is no correct object code for a model that cannot finish one evaluation, so the fix is a diagnostic rather than a substituted value.

Scope

These are integration nuances of running openvaf-r-compiled .osdi models in ngspice; they involve no bug and no pending fix. Verilog-A *language* coverage is tracked separately (the coverage audit), and the analog-only boundary — no wreal/digital nets — is covered there.